Claude Lahman Baseball Analysis

Colors:
Black = original instructions
Purple = my words
Blue = Claude's words
Green = table names
Red = formulas or SQL queries generated by Claude

In this exercise we will see how we can use Claude desktop as a source to help create SQL queries.

1. Start by passing the Claude desktop app a copy of the Lahman ERD. Ask for an interpretation of the ERD if it does not give one.
    a. I simply uploaded the ERD and Claude gave this response:
        i. This is the ER diagram for the Lahman Baseball database schema you have attached — no action needed unless you want me to do something with it. Want me to query this dataset, build a chart, or explore something specific?
    b. I asked Claude to interpret the ERD:
        i. This is the entity-relationship diagram for the Lahman Baseball database, a well-known historical dataset covering MLB players, teams, and statistics going back to the 1870s. Here's how it's structured.
            1. **People is the hub.** The `people` table (keyed on `playerid`) sits at the center and fans out to nearly every player-level table: `batting`, `pitching`, `fielding` (plus the split-out `fieldingof` and `fieldingofsplit` for outfield detail), `appearances`, `salaries`, `allstarfull`, `halloffame`, `collegeplaying`, and the various awards tables (`awardsplayers`, `awardsshareplayers`, `awardsmanagers`, `awardsshareplayers`). It also links to `managers` and `managershalf` for players who managed. Most of these child tables share a composite key pattern of `playerid` + `yearid` (+ `teamid`/`stint` where a player could appear on multiple teams or stints within a season).
            2. **Teams is the other hub.** The `teams` table (keyed on `yearid`, `lgid`, `teamid`) holds full season-level franchise stats — record, runs, hits, attendance, etc. It connects to `teamsfranchises` (the stable franchise identity across relocations/renames), `teamshalf`

(split-season data, e.g. strike years), `managers`/`managershalf`, `appearances`, `salaries`, and `homegames` (via `park`).

3. **Supporting reference tables.** `schools` feeds `collegeplaying`, which links back to `people` — tracking which college a player attended. `parks` feeds `homegames`, which ties a park to a team's season-by-season attendance/games data.
4. **Design pattern to note.** Almost every relationship is one-to-many from a "dimension" table (people, teams, parks, schools) out to yearly "fact" tables, and most fact tables use compound keys combining an entity ID with `yearid` (and sometimes `stint`, `half`, or `teamid`/`lgid`) to handle players/teams changing affiliations mid-season. This is a classic star-ish schema rather than pure normalization — `teamid`/`lgid` get repeated across many tables instead of only referencing `teams` once.
5. If you want, I can pull actual data from this and run some analysis or build a query.

c. Out of curiosity I asked it to “build a chart”. It asked me what kind and I said “no preference”. It automatically understood to fetch the data from github (I assume because its connected to the NSS teams account). This took way too long and required me to prompt it to “allow on this website” but eventually it gave me this:

i. I dug into this and hit a wall worth flagging before going further: this project only has the ERD schema image attached, not the actual Lahman data (no CSVs, no database file). I tried pulling the public dataset from GitHub to chart real numbers, but the original `chadwickbureau/baseballdatabank` repo is gone, and the mirrors I found are either unreachable or return incomplete/suspect data (one "Teams.csv" I pulled cut off around 1885, which isn't right).

ii. Given your project notes say never to fabricate data that isn't verifiable, I don't want to chart numbers I can't confirm are accurate.

iii. Easiest fix: if you have the actual Lahman CSVs or database file, upload them (or connect the folder they're in) and I'll build the chart from the real data — home runs by year, team wins, a player's career line, whatever you want. Want to do that?

2. Write a prompt to ask Claude which tables would be necessary to learn the first and last name of every manager who ever won the TSN Manager of the Year award for both the NL and AL including the teams they were representing for that year.

a. From the ERD alone, three tables suffice:

i. **awardsmanagers** — filter `awardid = 'TSN Manager of the Year'`, gives you `playerid`, `yearid`, `lgid` for each winner (both NL and AL, since lgid distinguishes them).

ii. **managers** — join on `playerid` + `yearid` + `lgid` to pull the `teamid` that manager represented that season (awardsmanagers itself has no team field).
iii. **people** — join on `playerid` to get `namefirst` and `namelast`.
iv. Optional: **teams** (join on `yearid+lgid+teamid`) if you want the full franchise name instead of just the team code.

3. Claude should tell you will need the awards manager, manager, and people tables at least. Write a simple query for these 3 tables one at a time and download the realated csv. Rename the csv to match the table they represent. Set these csv aside for now.
    a. Since it suggested teams as well, I also downloaded that one. Saved in my desktop AI Analytics folder (in the lahman data subfolder)

4. Write a prompt to instruct Claude to write a PostGreSQL query to answer the above question. Cut and paste that query into the query tool in PGAdmin. Look at the results. You should have 6 lines of output. If you have more or less, the query was not working properly. Examine, the output and query, see where there was a misunderstanding. Try to expand on your prompt or write a new one to get a better answer. Keep doing this until you get a proper output or you manually fix the query.

Claude's Initial SQL:

```
SELECT
    am.yearid                AS year,
    am.lgid                  AS league,
    p.namefirst              AS first_name,
    p.namelast               AS last_name,
    m.teamid                 AS team_id,
    t.name                   AS team_name
FROM awardsmanagers am
JOIN managers m
    ON m.playerid = am.playerid
   AND m.yearid   = am.yearid
   AND m.lgid     = am.lgid
JOIN people p
    ON p.playerid = am.playerid
LEFT JOIN teams t
    ON t.yearid = am.yearid
   AND t.lgid   = am.lgid
   AND t.teamid = m.teamid
WHERE am.awardid = 'TSN Manager of the Year'
ORDER BY am.yearid, am.lgid;
```

Logic:

The real execution order is: `FROM` → `JOIN` → `JOIN` → `LEFT JOIN` → `WHERE` → `SELECT` → `ORDER BY` — notably `WHERE` runs before `SELECT`, even though it's written after it, and `ORDER BY` runs dead last regardless of where it appears. Here is a breakdown:

```
FROM awardsmanagers am
```

Step 1. Postgres starts by materializing the base table, awardsmanagers, aliased am. At this point it's just every row in that table — no filtering yet.

```
JOIN managers m
   ON m.playerid = am.playerid
  AND m.yearid   = am.yearid
  AND m.lgid     = am.lgid
```

Step 2. For every row from step 1, Postgres looks for matching rows in `managers` on all three keys (`playerid`, `yearid`, `lgid`). Non-matches are dropped (inner join), matches are combined into wider rows. This is where the manager's `teamid` gets attached.

```
JOIN people p
   ON p.playerid = am.playerid
```

Step 3. Takes the result of step 2 and joins in `people` on `playerid`, attaching `namefirst`/`namelast`. Again, an inner join — any award-winner without a `people` row (shouldn't happen, but in principle) would get dropped here.

```
LEFT JOIN teams t
   ON t.yearid = am.yearid
  AND t.lgid   = am.lgid
  AND t.teamid = m.teamid
```

Step 4. Takes the result of step 3 and looks for a matching row in `teams` on year+league+team. Because this is a `LEFT JOIN`, rows from step 3 are kept even if no `teams` row matches — `team_name` would just come back `NULL` in that case instead of dropping the row.

```
WHERE am.awardid = 'TSN Manager of the Year'
```

Step 5. Only now does Postgres filter the combined rows down to just the ones where the award is TSN Manager of the Year. This runs *after* all the joins — Postgres built the full joined set (all awards, all managers) before throwing away everything that isn't this specific award.

```
SELECT
   am.yearid   AS year,
   am.lgid     AS league,
```

```
    p.namefirst AS first_name,
    p.namelast  AS last_name,
    m.teamid    AS team_id,
    t.name      AS team_name
```

Step 6. With the correct rows isolated, Postgres now projects just these six columns and applies the AS aliases — this is also when the column names you see in the output are actually assigned.

```
ORDER BY am.yearid, am.lgid;
```

Step 7. Last, the final result set is sorted by year, then league within each year. Sorting always happens last since it operates on the finished row set, not on intermediate joins.

The logic seems sound and I cannot for the life of me figure out why it should output only 6 lines instead of 60. I validated that there are exactly 60 awards given across the entire data set for those combined leagues (NL and AL).

5. Start a new chat (upper left) on Claude desktop. This time we will pass it 3 text descriptions of the tables.

6. In PGAdmin right-click the Lahman database and select the PSQL tool. In the interface type the command '\d people', this should output a text description of the people table. Just copy this output and paste it into the Claude desktop. Do the same for the managers and awardsmangers tables.

lahman_baseball-# \d people:

```
                      Table "public.people"
   Column    |         Type          | Collation | Nullable |       Default
-------------+-----------------------+-----------+----------+-----------------------
 playerid    | character varying(10) |           | not null | NULL::character varying
 birthyear   | integer               |           |          |
 birthmonth  | integer               |           |          |
 birthday    | integer               |           |          |
 birthcountry | character varying(50) |          |          | NULL::character varying
 birthstate  | character varying(50) |           |          | NULL::character varying
 birthcity   | character varying(50) |           |          | NULL::character varying
 deathyear   | integer               |           |          |
 deathmonth  | integer               |           |          |
 deathday    | integer               |           |          |
 deathcountry | character varying(50) |          |          | NULL::character varying
 deathstate  | character varying(50) |           |          | NULL::character varying
```

lahman_baseball=# \d managers:

```
                        Table "public.managers"
  Column  |         Type          | Collation | Nullable |         Default
----------+-----------------------+-----------+----------+-------------------------
 playerid | character varying(10) |           |          | NULL::character varying
 yearid   | integer               |           | not null |
 teamid   | character varying(3)  |           | not null |
 lgid     | character varying(2)  |           |          | NULL::character varying
 inseason | integer               |           | not null |
 g        | integer               |           |          |
 w        | integer               |           |          |
 l        | integer               |           |          |
 rank     | integer               |           |          |
 plyrmgr  | character varying(1)  |           |          | NULL::character varying
Indexes:
    "managers_pkey" PRIMARY KEY, btree (yearid, teamid, inseason)
```

lahman_baseball=# \d awardsmanagers:

```
                      Table "public.awardsmanagers"
  Column  |         Type          | Collation | Nullable |         Default
----------+-----------------------+-----------+----------+-------------------------
 playerid | character varying(10) |           | not null |
 awardid  | character varying(25) |           | not null |
 yearid   | integer               |           | not null |
 lgid     | character varying(2)  |           | not null |
 tie      | character varying(1)  |           |          | NULL::character varying
 notes    | character varying(100) |          |          | NULL::character varying
Indexes:
    "awardsmanagers_pkey" PRIMARY KEY, btree (yearid, awardid, lgid, playerid)
```

7. Once again ask Claude to write a SQL query to answer question 2. Was there an difference in the two queries.
    a. My prompt: Write a sql query to output the first and last name of every manager who ever won the TSN Manager of the Year award for BOTH the NL and AL including the teams they were representing for that year (only include ones that won at least once in both leagues)
    b. Claude's answer is below, Note: It did a very different query this time. It did repeat the mistake with the count of league id =2 but when i corrected it, it came up with a different answer than before (bool_or was used last time, a CTE was

used this time). The result was still 6 rows and the same names (Davey Johnson and Jim Leyland)

```
WITH tsn_wins AS (
    SELECT playerid, yearid, lgid
    FROM awardsmanagers
    WHERE awardid = 'TSN Manager of the Year'
),
both_leagues AS (
    SELECT playerid
    FROM tsn_wins
    GROUP BY playerid
    HAVING COUNT(DISTINCT CASE WHEN lgid = 'AL' THEN 1 END) > 0
       AND COUNT(DISTINCT CASE WHEN lgid = 'NL' THEN 1 END) > 0
)
SELECT
    p.namefirst,
    p.namelast,
    w.yearid,
    w.lgid,
    m.teamid
FROM tsn_wins w
JOIN both_leagues b ON b.playerid = w.playerid
JOIN people p ON p.playerid = w.playerid
JOIN managers m
    ON m.playerid = w.playerid
   AND m.yearid = w.yearid
   AND m.lgid = w.lgid
WHERE w.lgid IN ('AL', 'NL')
ORDER BY p.namelast, p.namefirst, w.yearid;
```

I asked claude to explain the code in order of execution:

Order of execution, step by step:

1. **`tsn_wins` CTE runs first.** It scans `awardsmanagers`, keeps only rows where `awardid = 'TSN Manager of the Year'`, and returns `(playerid, yearid, lgid)` for every such win — AL, NL, ML, whatever. Nothing is excluded by league yet.
2. **`both_leagues` CTE consumes that result.** It groups those award rows by `playerid`. Within each group, two conditional counts are computed: `COUNT(DISTINCT CASE WHEN lgid='AL' THEN 1 END)` and the same for `'NL'`. A `CASE` with no `ELSE` returns `NULL` for non-matching rows, and `COUNT` ignores `NULL`s — so each expression only counts rows that actually match that league. `HAVING` then keeps a `playerid` only if both counts are `> 0`, i.e. the manager has at least one AL win and at least one NL win

somewhere in their full award history (a third league present or not doesn't affect this test). Output is just a list of qualifying `playerid`s.

3. **Main query's** `FROM tsn_wins w` starts again from the full award-win list (every league, same as step 1).
4. `JOIN both_leagues b` filters that list down to only rows belonging to a qualifying manager — this is where the "both leagues" requirement actually gets enforced on the output rows.
5. `JOIN people p` attaches `namefirst`/`namelast` for those rows.
6. `JOIN managers m` matches on `playerid + yearid + lgid` to pull the team(s) that manager led in that specific award-winning year/league (can return more than one row if they changed teams mid-season).
7. `WHERE w.lgid IN ('AL','NL')` runs last among the filters, on the joined rows. This trims away any leftover ML (or other league) award rows from a qualifying manager's output — it does not affect who qualifies, only which rows get printed.
8. `SELECT` picks the final columns, then `ORDER BY` sorts by last name, first name, year.

The key logic: qualification (step 2) is decided from the *entire* award history per manager, independent of league count; display filtering (step 7) is a separate, later step that only trims which rows of a qualifying manager show up.

8. Now open the people table in Excel. You can add files to the existing workbook through Claude for Excel. Add the manager csv as a new sheet then do the same for awardsmanager. Prompt Claude to create a new sheet that has a list of the managers the meet the criteria mentioned above. Again, there should be 6 lines of output in the workbook. If it did not work as expected have Claude update the output to match the actual answer.
    a. I had to use the claude browser in google sheets since i do not have excel on my macbook.
    b. Surprisingly it actually returned the right answer the first time with no additional prompting. I didn't even have to remind it that more than two leagues are present in the data (I assume since it can see the data directly, unlike the previous two times where it can only see the ERD or the table descriptions). These are the exact same names surfaced in the other two queries.
    c. I also asked it to explain its logic/any formulas it used. One downside to using claude with google sheets (or excel) is that the way it arrived at its answer is not always clear. You have to prompt it to provide this information to make sure its verifiable.

Done. Only two managers in the data ever won the TSN Manager of the Year award in both the AL and the NL, and here's the breakdown with the teams they were managing in each winning year:

First Name	Last Name	League	Year	Team
Davey	Johnson	AL	1997	Baltimore Orioles
Davey	Johnson	NL	2012	Washington Nationals
Jim	Leyland	NL	1988	Pittsburgh Pirates
Jim	Leyland	NL	1990	Pittsburgh Pirates
Jim	Leyland	NL	1992	Pittsburgh Pirates
Jim	Leyland	AL	2006	Detroit Tigers

A couple of notes on how I approached this:

The award data included early years (1936–1951) where TSN gave a single "Major League" (ML) award not split by league, plus a separate "BBWAA Manager of the Year" award. I only counted actual AL and NL wins toward the "both leagues" requirement, since the question asks specifically about winning in both the National and American Leagues. On that basis, Davey Johnson and Jim Leyland are the only two who qualified.

I listed each winning year separately so you can see the teams — Leyland won three times in the NL with Pittsburgh before his AL win with Detroit, and Johnson won once in each league. If you'd prefer this condensed to one row per manager (e.g., "Jim Leyland — NL: Pirates; AL: Tigers"), or want me to add the BBWAA award as a separate consideration, just let me know.

Here's the full walkthrough of how I found this, in the order I worked through it.

First, I explored the data structure. I didn't know which columns held what, so I wrote some temporary "scratch" formulas off to the side (columns O–Z) to inspect things. To find the distinct award names, I used `=TEXTJOIN("|",TRUE,UNIQUE(awardsmanagers!B2:B))`, which returned "BBWAA Manager of the Year|TSN Manager of the Year". A similar formula on the league column returned "AL|NL|ML", telling me leagues appeared as AL, NL, or ML (the ML being the old single Major League award before it split by league). I also pulled the header rows of each sheet with `TEXTJOIN` and used `MATCH` (e.g. `=MATCH("namefirst",people!1:1,0)`) to find exact column positions, which told me people's first/last names were in columns N and O, and the teams sheet had its team name in column AO, teamid in C, year in A, and league in B.

Second, I identified the managers who won in both leagues. This was the core logic. I used this formula:

`=UNIQUE(FILTER(awardsmanagers!A2:A, awardsmanagers!B2:B="TSN Manager of the Year", awardsmanagers!D2:D="AL", COUNTIFS(awardsmanagers!A2:A, awardsmanagers!A2:A, awardsmanagers!B2:B, "TSN Manager of the Year", awardsmanagers!D2:D, "NL")>0))`

The logic here: filter down to TSN winners in the AL, but only keep rows where that same playerid also has a `COUNTIFS` count greater than zero for a TSN win in the NL. Requiring an AL win AND a matching NL win guarantees both leagues. That returned two player IDs: `johnsda02` and `leylaji99`.

Third, I pulled every winning year for those two. I used `=SORT(FILTER({awardsmanagers!A2:A, awardsmanagers!C2:C, awardsmanagers!D2:D}, awardsmanagers!B2:B="TSN Manager of the Year", (awardsmanagers!A2:A="johnsda02")+(awardsmanagers!A2:A="leylaji99")))` to get playerid, year, and league for each award. The `(...)+( ...)` construct acts as an OR so I captured both managers at once.

Fourth, I resolved the names and teams. For names I did a `VLOOKUP` of the playerid against the people sheet's first-name and last-name columns. The tricky part was teams, because the awards table has no team column. So I matched playerid + year + league against the managers sheet to get the team code (`=FILTER(managers!C:C, managers!A:A=playerid, managers!B:B=year, managers!D:D=league)`), which gave codes like BAL, WAS, PIT, DET. Then I converted those codes to full team names by matching year + teamid against the teams sheet's name column (`=FILTER(teams!AO:AO, teams!A:A=year, teams!C:C=teamcode)`).

Finally, I read off the resolved values, deleted all the scratch formulas, and typed the final clean table into A1:E7.

One judgment call worth flagging: I deliberately excluded the old "ML" (single Major League) TSN awards from the both-leagues test, since those predate the AL/NL split and aren't league-specific. If you'd rather I treat the data differently, or want me to leave the working formulas in the sheet so you can audit them live, I'm happy to redo it that way.